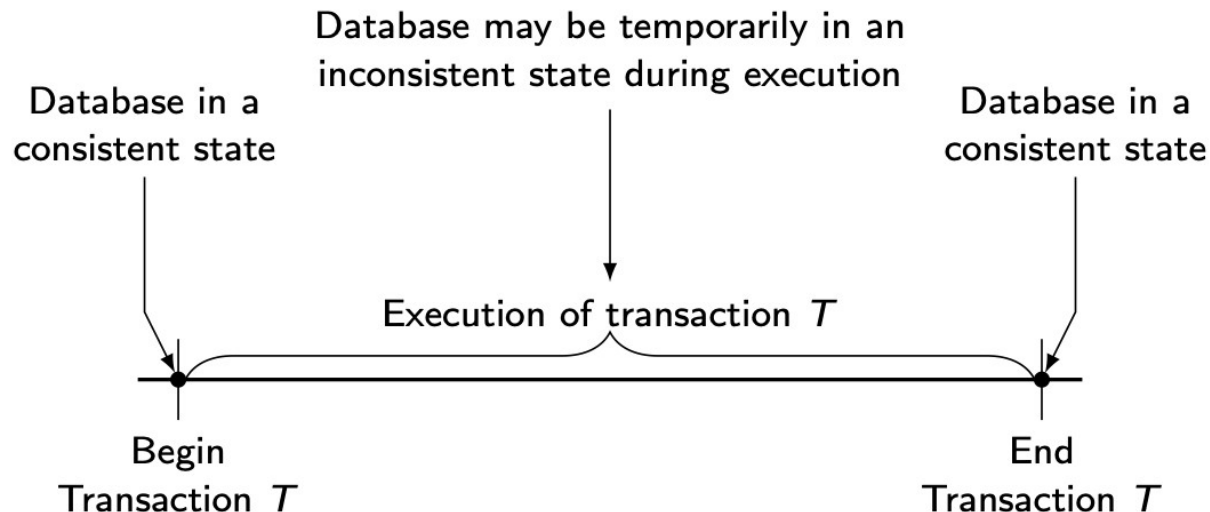


Transaction

A transaction is a collection of actions that make consistent transformations of system states while preserving system consistency.

- ❑ concurrency transparency
- ❑ failure transparency



Transaction Characterization

Begin_transaction

...

Read

Read

...

Write

Read

...

Commit

- **Read set (RS)**
 - ▣ The set of data items that are read by a transaction
- **Write set (WS)**
 - ▣ The set of data items whose values are changed by this transaction
- **Base set (BS)**
 - ▣ $RS \cup WS$

Principles of Transactions

ATOMICITY

- ❑ all or nothing

CONSISTENCY

- ❑ no violation of integrity constraints

ISOLATION

- ❑ concurrent changes invisible $\Rightarrow$ serializable

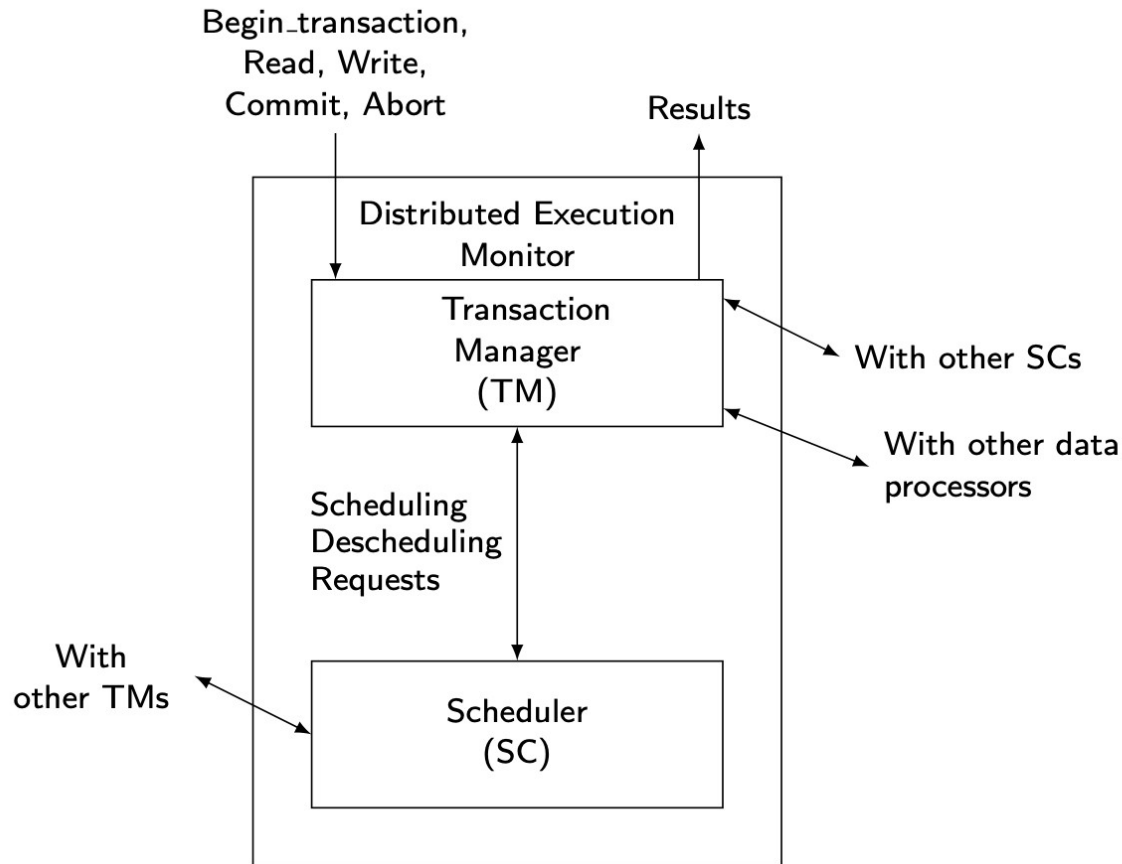
DURABILITY

- ❑ committed updates persist

Transactions Provide...

- *Atomic* and *reliable* execution in the presence of failures
- *Correct* execution in the presence of multiple user accesses
- Correct management of *replicas* (if they support it)

Distributed TM Architecture



Outline

- Distributed Transaction Processing
 - ▣ Distributed Concurrency Control
 - ▣ Distributed Reliability

Concurrency Control

- The problem of synchronizing concurrent transactions such that the consistency of the database is maintained while, at the same time, maximum degree of concurrency is achieved.
- Enforce **isolation** property
- Anomalies:
 - Lost updates
 - The effects of some transactions are not reflected on the database.
 - Inconsistent retrievals
 - A transaction, if it reads the same data item more than once, should always read the same value.

Serializability in Distributed DBMS

- Two histories have to be considered:
 - local histories
 - global history
- For global transactions (i.e., global history) to be **serializable**, two conditions are necessary:
 - Each local history should be serializable → **local serializability**
 - Two conflicting operations should be in the same relative order in all of the local histories where they appear together → **global serializability**

Global Non-serializability

T_1 :	Read(x) $x \leftarrow x - 100$ Read(y) Write(x) Commit Read(y) $y \leftarrow y + 100$ Write(y) Commit	T_2 :	Read(x)
---------	---	---------	---------

- x stored at Site 1, y stored at Site 2
- LH_1 , LH_2 are individually serializable (in fact serial), but the two transactions are not globally serializable.

$$LH_1 = \{R_1(x), W_1(x), R_2(x)\}$$

$$LH_2 = \{R_2(y), R_1(y), W_1(y)\}$$

Concurrency Control Algorithms

- Pessimistic
 - Two-Phase Locking-based (2PL)
 - Centralized (primary site) 2PL
 - Distributed 2PL
 - Timestamp Ordering (TO)
 - Basic TO
 - Multiversion TO
 - Conservative TO
- Optimistic
 - Locking-based
 - Timestamp ordering-based

Locking-Based Algorithms

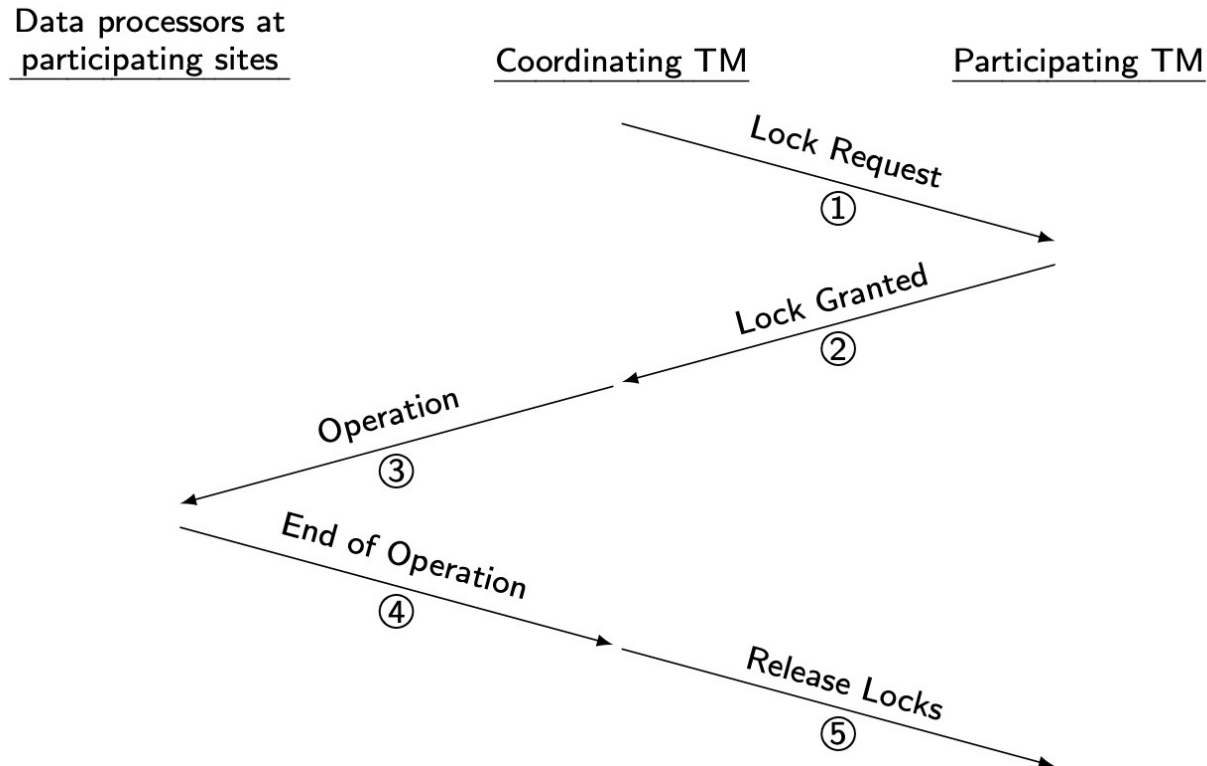
- Transactions indicate their intentions by requesting locks from the scheduler (called **lock manager**).
- Locks are either **read lock** (*rl*) [also called **shared lock**] or **write lock** (*wl*) [also called **exclusive lock**]
- Read locks and write locks conflict (because Read and Write operations are incompatible)

	<i>rl</i>	<i>wl</i>
<i>rl</i>	yes	no
<i>wl</i>	no	no

- Locking works nicely to allow concurrent processing of transactions.

Centralized 2PL

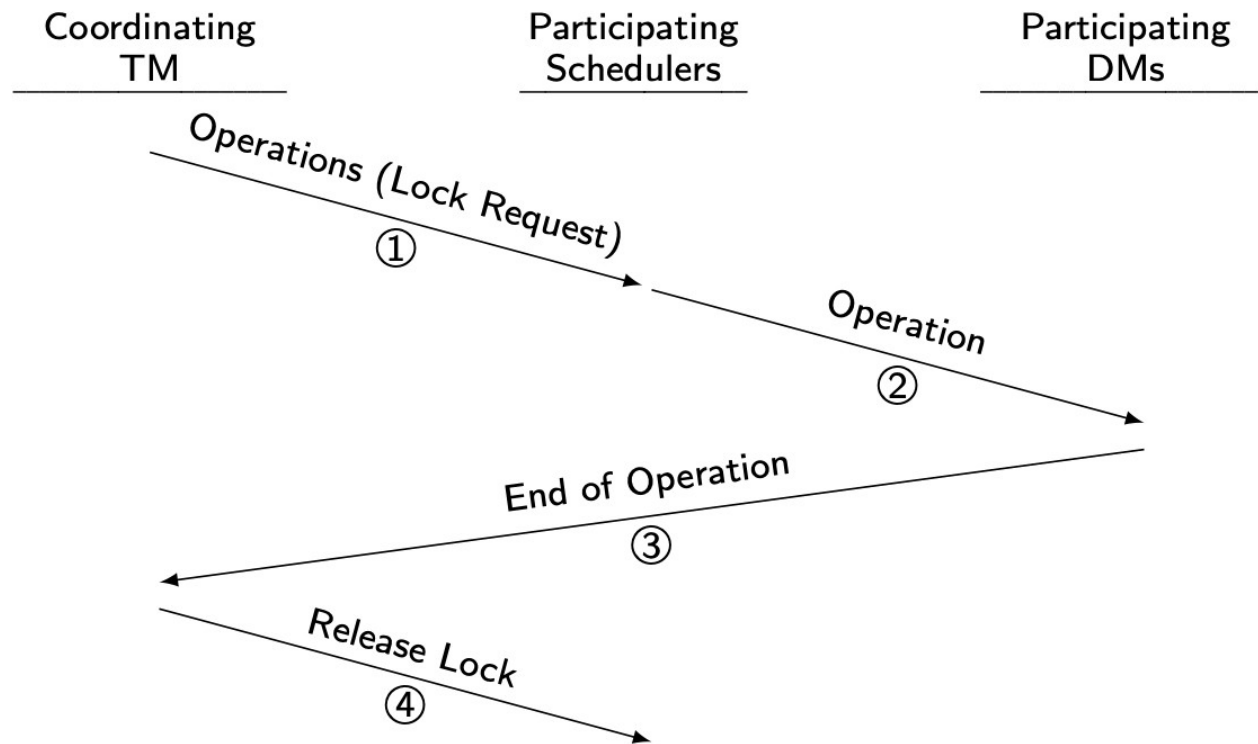
- There is only one 2PL scheduler in the distributed system.
- Lock requests are issued to the central scheduler.



Distributed 2PL

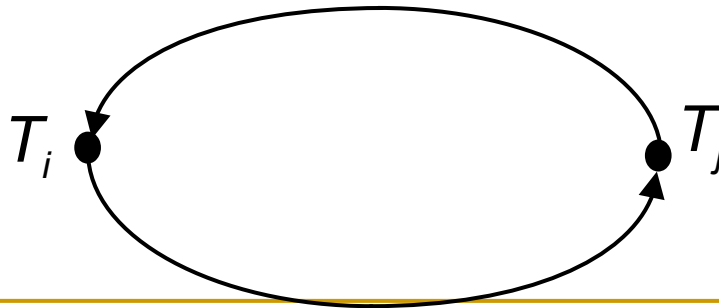
- 2PL schedulers are placed at each site. Each scheduler handles lock requests for data at that site.
- A transaction may read any of the replicated copies of item x , by obtaining a read lock on one of the copies of x . Writing into x requires obtaining write locks for all copies of x .

Distributed 2PL Execution



Deadlock

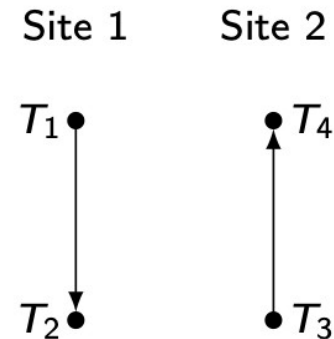
- A transaction is deadlocked if it is blocked and will remain blocked until there is intervention.
- Locking-based CC algorithms may cause deadlocks.
- TO-based algorithms that involve waiting may cause deadlocks.
- Wait-for graph
 - ▣ If transaction T_i waits for another transaction T_j to release a lock on an entity, then $T_i \rightarrow T_j$ in WFG.



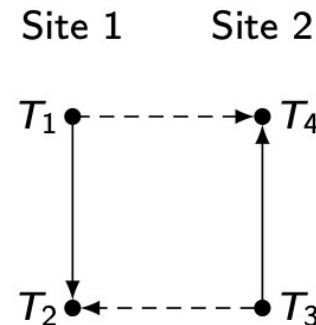
Local versus Global WFG

- T_1 and T_2 run at site 1, T_3 and T_4 run at site 2.
- T_3 waits for a lock held by T_4 which waits for a lock held by T_1 which waits for a lock held by T_2 which, in turn, waits for a lock held by T_3 .

Local WFG



Global WFG



Deadlock Detection

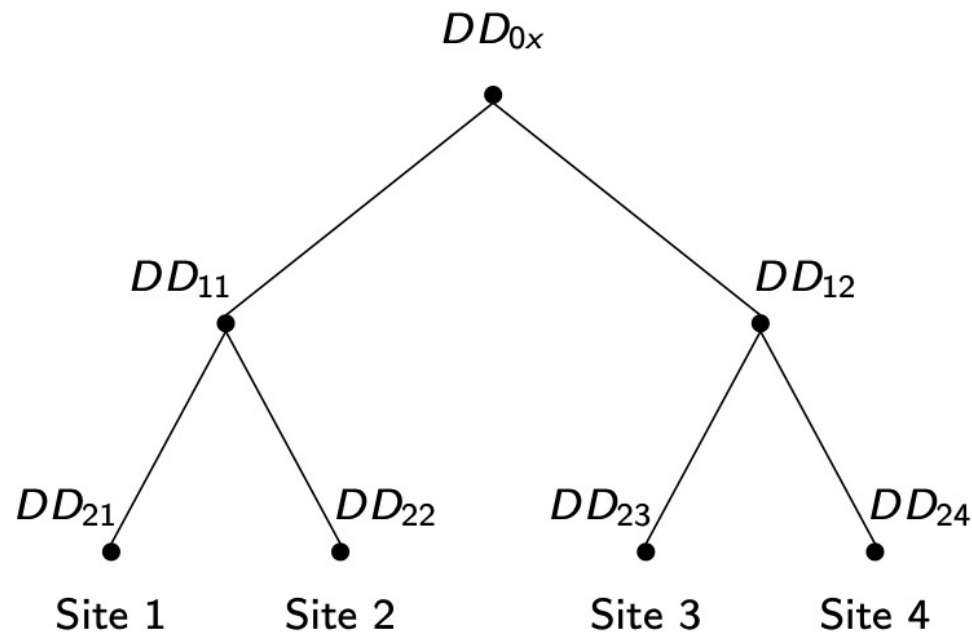
- Transactions are allowed to wait freely.
- Wait-for graphs and cycles.
- Topologies for deadlock detection algorithms
 - Centralized
 - Distributed
 - Hierarchical

Centralized Deadlock Detection

- One site is designated as the deadlock detector for the system. Each scheduler periodically sends its local WFG to the central site which merges them to a global WFG to determine cycles.
- How often to transmit?
 - Too often $\Rightarrow$ higher communication cost but lower delays due to undetected deadlocks
 - Too late $\Rightarrow$ higher delays due to deadlocks, but lower communication cost
- Would be a reasonable choice if the concurrency control algorithm is also centralized.
- Proposed for Distributed INGRES

Hierarchical Deadlock Detection

Build a hierarchy of detectors



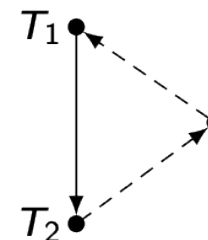
Distributed Deadlock Detection

- Sites cooperate in detection of deadlocks.
- One example:

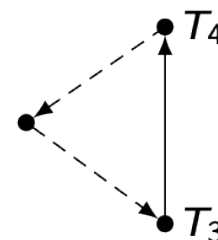
- Form local WFGs at each modified as follows:

- 1) Potential deadlock cycles from other sites are added as edges
- 2) Join these with regular edges
- 3) Pass these local WFGs to other sites

Site 1



Site 2



- Each local deadlock detector:

- looks for a cycle that does not involve the external edge. If it exists, there is a local deadlock which can be handled locally.
- looks for a cycle involving the external edge. If it exists, it indicates a **potential** global deadlock. Pass on the information to the next site.

Timestamp Ordering

- 1 Transaction (T_i) is assigned a globally unique timestamp $ts(T_i)$.
- 2 Transaction manager attaches the timestamp to all operations issued by the transaction.
- 3 Each data item is assigned a write timestamp (wts) and a read timestamp (rts):
 - ▢ $rts(x)$ = largest timestamp of any read on x
 - ▢ $wts(x)$ = largest timestamp of any read on x
- 4 Conflicting operations are resolved by timestamp order.

Basic T/O:

for $R_i(x)$ for $W_i(x)$

if $ts(T_i) < wts(x)$ **if** $ts(T_i) < rts(x)$ **and** $ts(T_i) < wts(x)$

then reject $R_i(x)$ **then** reject $W_i(x)$

else accept $R_i(x)$ **else** accept $W_i(x)$

$rts(x) \leftarrow ts(T_i)$ $wts(x) \leftarrow ts(T_i)$

Basic Timestamp Ordering

Two conflicting operations O_{ij} of T_i and O_{kl} of $T_k \rightarrow O_{ij}$ executed before O_{kl} iff $ts(T_i) < ts(T_k)$.

- ❑ T_i is called **older** transaction
- ❑ T_k is called **younger** transaction

for $R_i(x)$ for $W_i(x)$

if $ts(T_i) < wts(x)$ **if** $ts(T_i) < rts(x)$ **and** $ts(T_i) < wts(x)$

then reject $R_i(x)$

then reject $W_i(x)$

else accept $R_i(x)$

else accept $W_i(x)$

$rts(x) \leftarrow ts(T_i)$ $wts(x) \leftarrow ts(T_i)$

Conservative Timestamp Ordering

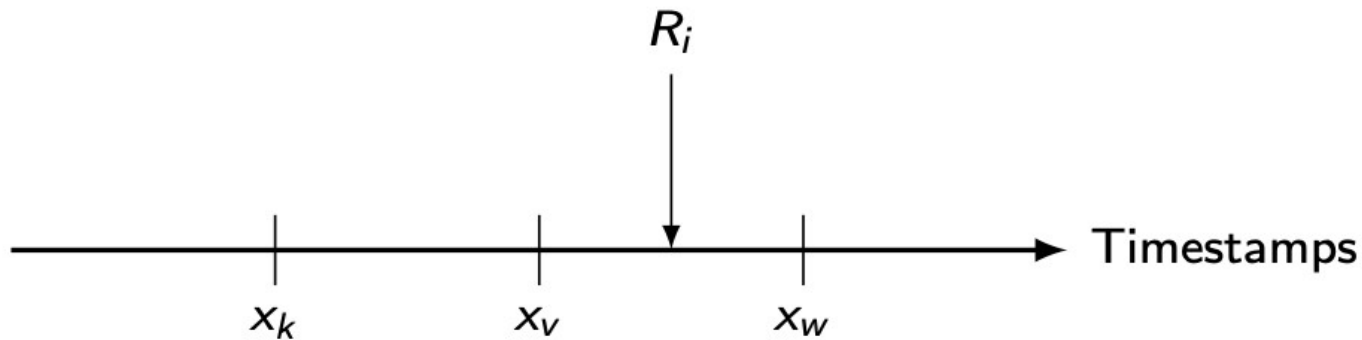
- Basic timestamp ordering tries to execute an operation as soon as it receives it
 - progressive
 - too many restarts since there is no delaying
- Conservative timestamping delays each operation until there is an assurance that it will not be restarted
- Assurance?
 - No other operation with a smaller timestamp can arrive at the scheduler
 - Note that the delay may result in the formation of deadlocks

Multiversion Concurrency Control (MVCC)

- Do not modify the values in the database, create new values.
- Typically timestamp-based implementation
$$ts(T_i) < ts(x_r) < ts(T_j)$$
- Implemented in a number of systems: IBM DB2, Oracle, SQL Server, SAP HANA, BerkeleyDB, PostgreSQL

MVCC Reads

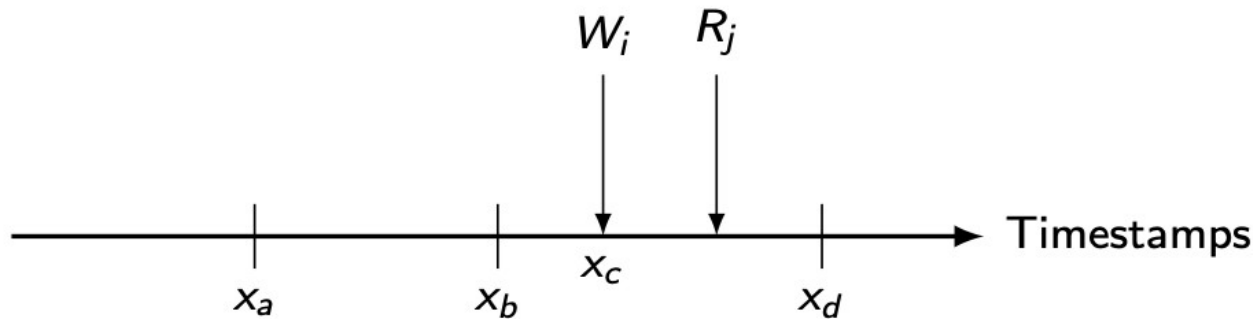
- A $R_i(x)$ is translated into a read on one version of x .
 - ▣ Find a version of x (say x_v) such that $ts(x_v)$ is the largest timestamp less than $ts(T_i)$.



MVCC Writes

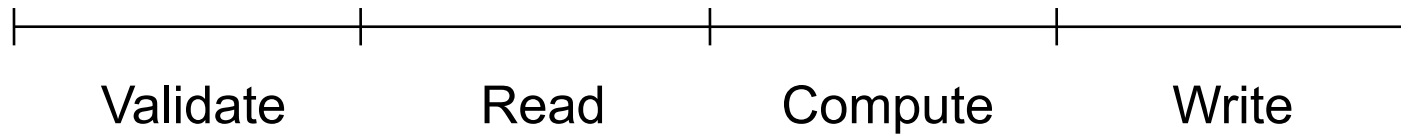
- A $W_i(x)$ is translated into $W_i(x_w)$ and accepted if the scheduler has not yet processed any $R_j(x_r)$ such that

$$ts(T_i) < ts(x_r) < ts(T_j)$$

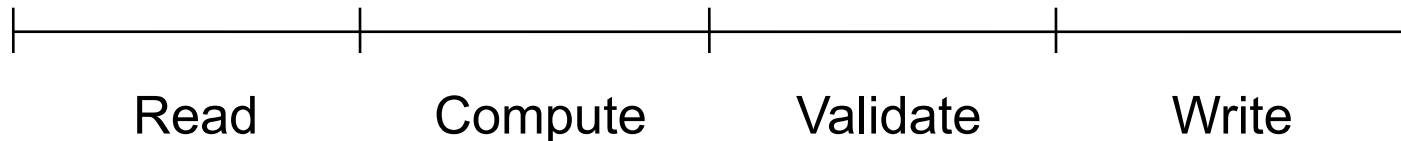


Optimistic Concurrency Control Algorithms

Pessimistic execution



Optimistic execution



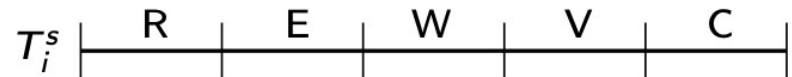
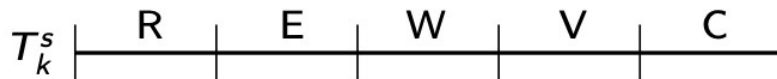
Optimistic Concurrency Control Algorithms

- Transaction execution model: divide into subtransactions each of which execute at a site
 - T_{ij} : transaction T_i that executes at site j
- Transactions run independently at each site until they reach the end of their read phases
- All subtransactions are assigned a timestamp at the end of their read phase
- **Validation test** performed during validation phase. If one fails, all rejected.

Optimistic CC Validation Test

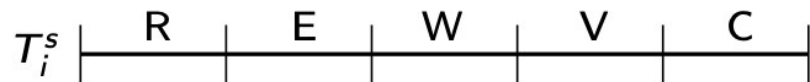
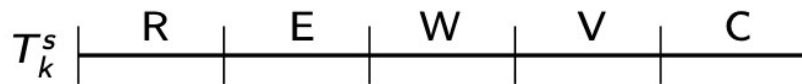
- 1 If all transactions T_k where $ts(T_k) < ts(T_{ij})$ have completed their write phase before T_{ij} has started its read phase, then validation succeeds

□ Transaction executions in serial order



Optimistic CC Validation Test

- ② If there is any transaction T_k such that $ts(T_k) < ts(T_{ij})$ and which completes its write phase while T_{ij} is in its read phase, then validation succeeds if $WS(T_k) \cap RS(T_{ij}) = \emptyset$
- Read and write phases overlap, but T_{ij} does not read data items written by T_k

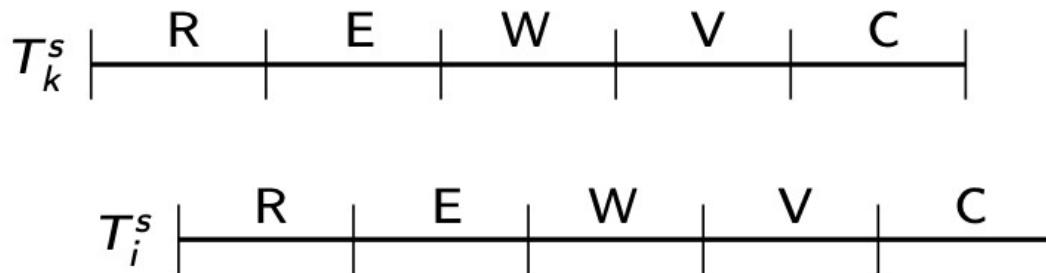


Optimistic CC Validation Test

- ③ If there is any transaction T_k such that $ts(T_k) < ts(T_{ij})$ and which completes its read phase before T_{ij} completes its read phase, then validation succeeds if

$$WS(T_k) \cap RS(T_{ij}) = \emptyset \text{ and } WS(T_k) \cap WS(T_{ij}) = \emptyset$$

- They overlap, but don't access any common data items.



Snapshot Isolation (SI)

- Each transaction “sees” a consistent snapshot of the database when it starts and R/W this snapshot
- Repeatable reads, but not serializable isolation
- Read-only transactions proceed without significant synchronization overhead
- Centralized SI-based CC
 - 1) T_i starts, obtains a begin timestamp $ts_b(T_i)$
 - 2) T_i ready to commit, obtains a commit timestamp $ts_c(T_i)$ that is greater than any of the existing ts_b or ts_c
 - 3) T_i commits if no other T_j such that $tsc(T_j) [ts_b(T_i), ts_c(T_i)]$; otherwise aborted (**first committer wins**)
 - 4) When T_i commits, changes visible to all T_k where $ts_b(T_k) > ts_c(T_i)$

Distributed CC with SI

- Computing a consistent **distributed** snapshot is hard
- Similar rules to serializability
 - Each local history should be SI
 - Global history is SI $\rightarrow$ **commitment orders** at each site are the same
- **Dependence relationship**: T_i at site s (T_i^s) is dependent on T_j^s ($dependent(T_i^s, T_j^s)$) iff
$$(RS(T_i^s) \cap WS(T_j^s) \neq \emptyset) \vee (WS(T_i^s) \cap RS(T_j^s) \neq \emptyset) \vee (WS(T_i^s) \cap WS(T_j^s) \neq \emptyset)$$
- **Conditions**
 - 1) $dependent(T_i, T_j) \wedge ts_b(T_i^s) < ts_c(T_j^s) \Rightarrow ts_b(T_i^t) < ts_c(T_j^t)$ at every site t where T_i and T_j execute together
 - 2) $dependent(T_i, T_j) \wedge ts_c(T_i^s) < ts_c(T_j^s) \Rightarrow ts_c(T_i^t) < ts_b(T_j^t)$ at every site t where T_i and T_j execute together
 - 3) $ts_c(T_i^s) < ts_c(T_j^s) \Rightarrow ts_c(T_i^t) < ts_b(T_j^t)$ at every site t where T_i and T_j execute together

Distributed CC with SI – Executing T_i

